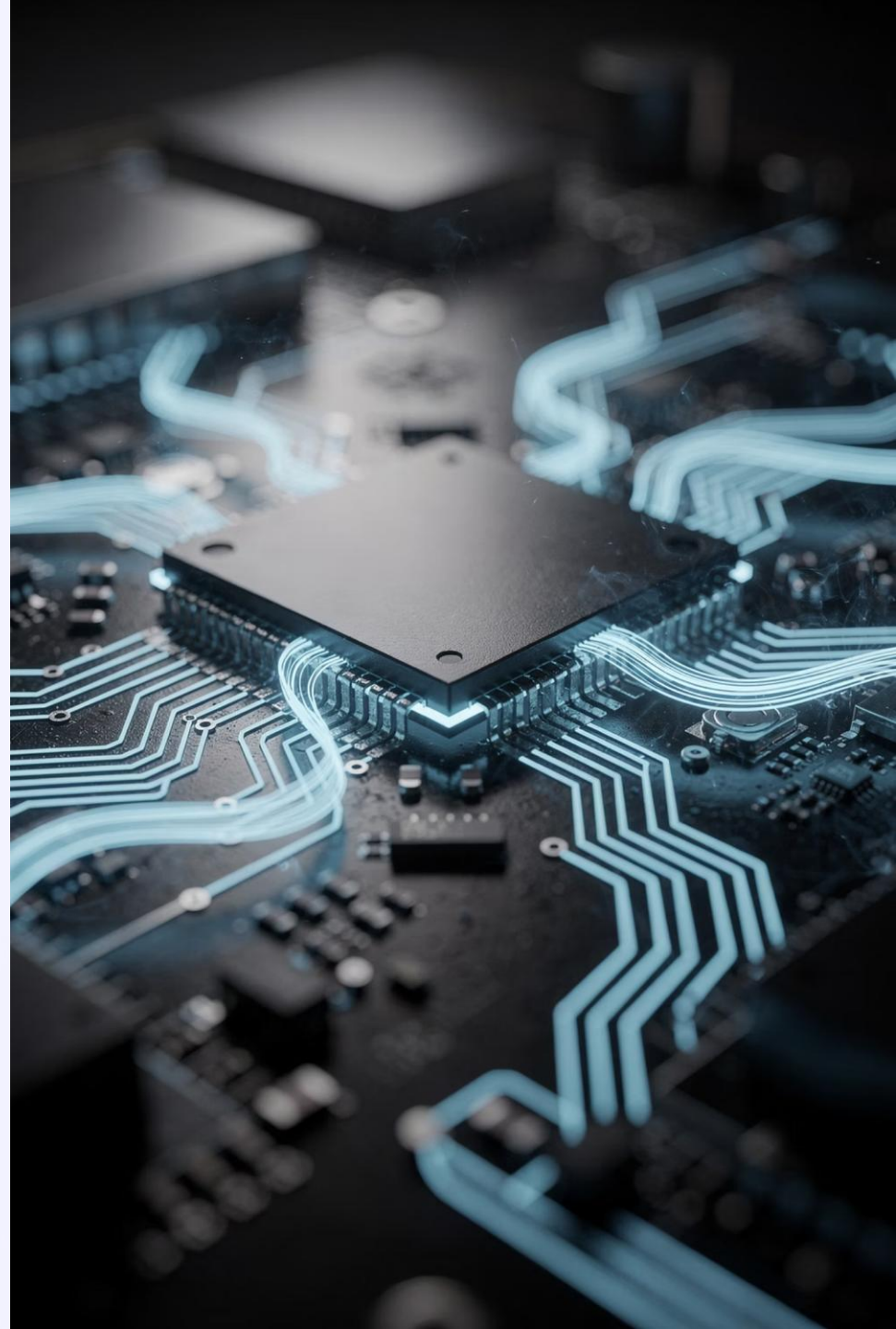


電腦演進史

從大型主機到 AI 時代

程式設計概論





想像一下：回到 1960 年

如果你生活在 1960 年代，你對「電腦」的認識，會和今天截然不同。

📏 體積

一台電腦比整間教室還大，需要專屬機房與空調系統才能運作。

👥 使用方式

幾十人共同輪流使用同一台機器，需要預約排班，等待時間長達數小時。

💰 成本

造價可能高達上億元，只有政府機關或大型企業才負擔得起。

🚫 可及性

一般民眾幾乎不可能接觸，更遑論擁有一台屬於自己的電腦。

❓ 問題思考：你覺得當時有可能把電腦放進口袋嗎？是什麼讓一切改變了？

電腦演進的四大核心趨勢

幾十年來，電腦技術沿著四個方向持續突破，每一個方向的進步都互相強化，共同推動了這場改變世界的科技革命。



體積越來越小

從佔據整棟建築，到放入口袋，電腦的物理尺寸不斷縮小。



成本越來越低

從上億元的機器，到百元的微控制器，技術普及讓價格大幅下降。



運算速度越來越快

處理能力以指數型成長，今日的手機已超越昔日的超級電腦。



功能越來越強

從單純計算數字，到語音辨識、影像生成、自然語言理解，無所不能。

💡 問題思考：是什麼驅動了這四個趨勢？接下來我們來看關鍵答案。

第一階段

大型主機（Mainframe）時代

核心特色

- 體積巨大，佔據整個機房
- 集中式運算架構
- 多人共用同一台主機
- 需要專業操作人員維護

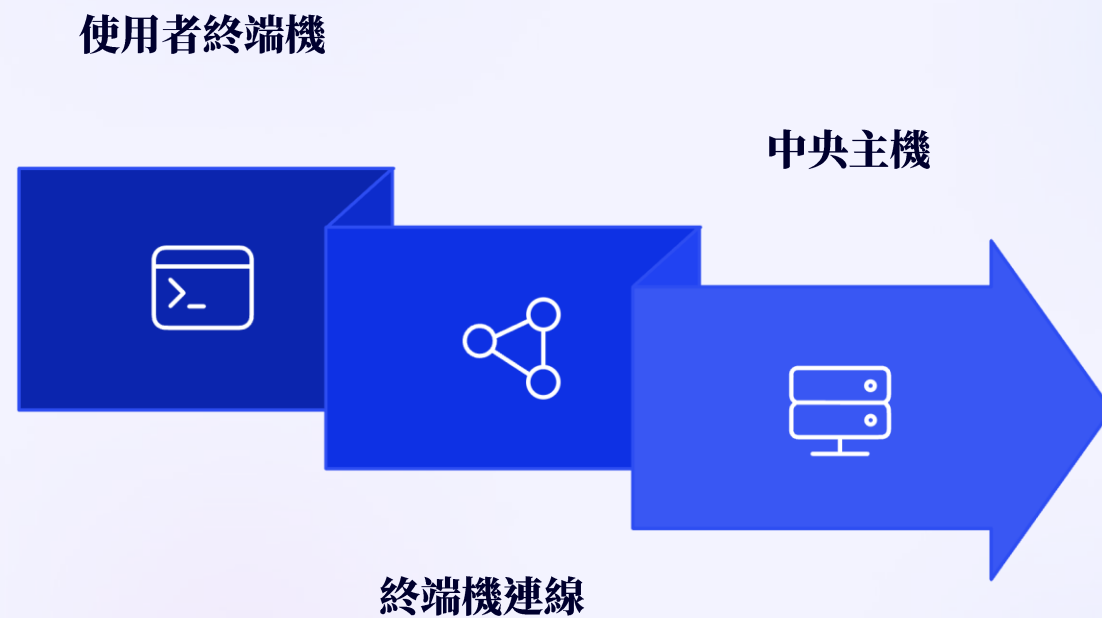
主要應用場景

- 政府機關：人口普查、稅務系統
- 銀行金融：帳戶管理、資金清算
- 軍事國防：彈道計算、密碼破解
- 大型研究機構：科學模擬計算



Mainframe 的使用方式

在大型主機時代，電腦資源極為稀缺，使用者必須透過「終端機」遠端連接到中央主機，所有的運算都在那一台巨大的機器上完成。



這種架構稱為「集中式運算」——所有人共用一台超級電腦。使用者沒有自己的運算能力，完全依賴主機的資源配置。這個模式雖然有效率，卻也造成了巨大的使用瓶頸。

莫爾定律 (Moore's Law)

「晶片上的電晶體數量，大約每 18 ~ 24 個月就會增加一倍。」

— Gordon Moore · Intel 共同創辦人，1965 年

⚡ 更快

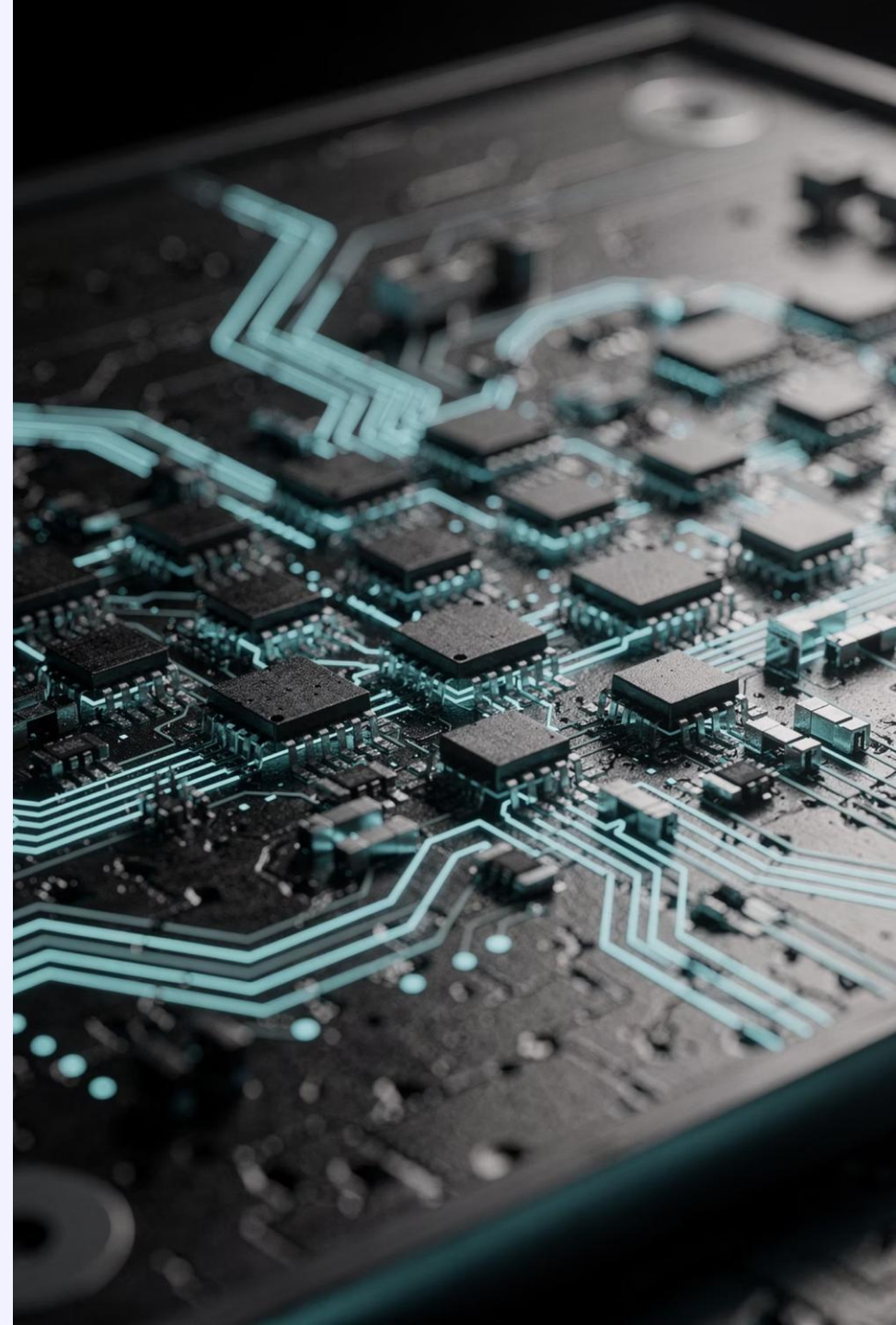
電晶體數量倍增，處理器時脈頻率隨之提升，運算速度指數型加快。

🔬 更小

製程技術不斷精進，從微米到奈米，晶片體積持續縮小。

💰 更便宜

量產規模擴大與技術成熟，使每單位運算的成本大幅降低。



莫爾定律帶來的驚人改變

莫爾定律不只是一條觀察規律，它是推動整個電腦演進史的核心引擎。硬體能力的指數型成長，讓「不可能」一再變成「理所當然」。

以前（1960年代）

一台電腦重達數公噸，體積填滿整個房間，造價折合現今超過億元，且只能完成特定計算任務，由少數專家操作。

現在（2020年代）

你口袋裡的智慧型手機，運算能力已遠超過 1960 年代的超級電腦，售價不過數萬元，且能執行 AI、3D 遊戲、即時翻譯等複雜工作。

  **指數型成長的力量：**每 2 年翻倍，30 年後就是原來的 **32,768 倍**。這就是為什麼科技進步讓人感覺「突然加速」。



第二階段

個人電腦（PC）時代



時代代表

- Apple II (1977) : 最早普及的家用電腦
- IBM PC (1981) : 確立業界標準架構
- Microsoft Windows (1985) : 圖形化介面普及



革命性的改變

- 電腦從機房走進家庭與辦公室
- 人人都能擁有屬於自己的電腦
- 軟體產業開始蓬勃發展
- 文書處理、試算表改變工作方式

PC 革命：從共享到個人

PC 時代最根本的變革，不只是技術進步，而是改變了人與電腦之間的關係。



大型主機時代

多人共享一台電腦

使用者需要排隊等候，透過終端機輸入指令，等待中央主機回應。
電腦是稀缺資源，使用時間被嚴格分配。



個人電腦時代

一人擁有一台電腦

每個人都有自己的運算能力，隨時可以使用，不需要等待。電腦成為個人生產力工具，普及率快速攀升。



影響：1980 年代末期，美國家庭電腦擁有率從近乎 0% 快速成長，開啟了軟體、遊戲、數位內容的全新產業生態。

第三階段

網路時代：電腦開始互相連接

🌐 關鍵技術突破

- **Internet**：全球電腦網路基礎建設
- **E-mail**：取代傳統郵件的數位通訊
- **World Wide Web (1991)**：網頁與超連結
- **瀏覽器**：讓一般人能輕鬆上網

💻 Client / Server 架構

網路時代帶來了新的運算架構：

- **Client (客戶端)**：你的電腦，負責顯示
- **Server (伺服器)**：遠端主機，負責儲存與處理

這是「去中心化」與「分散式運算」的開端。



網路世界的誕生：資料開始流動

網路時代最深刻的改變，是讓原本孤立的電腦資料「活」了起來——不再被鎖在單一裝置，而是在全球各地流動、連結、被分享。



Yahoo (1995)

最早的網路目錄與入口網站，讓人們第一次有系統地瀏覽網際網路上的資訊。



Google (1998)

革命性的搜尋引擎，用 PageRank 演算法讓資訊檢索變得精準高效，徹底改變人們獲取知識的方式。



MSN Messenger (1999)

即時通訊軟體讓網路從「查資料」演進為「與人連結」，開啟了社群溝通的新時代。

行動裝置時代：電腦放進口袋

還記得第一堂課的問題嗎？「你覺得有可能把電腦放進口袋嗎？」——答案在 2007 年揭曉了。



Smartphone 智慧型手機

iPhone (2007) 重新定義了手機，將電話、相機、音樂播放器、瀏覽器合而為一，開創 App 生態系。



Tablet 平板電腦

iPad (2010) 填補了手機與筆電之間的空白，為教育、設計、娛樂提供了全新的互動介面。



隨時隨地上網

3G/4G/5G 行動網路讓「連線」不再受限於地點，電腦的使用情境從桌前延伸到生活的每一個角落。





第五階段

雲端時代：資料不再綁定設備

☁ 雲端的核心特色

- 資料儲存在遠端網路伺服器
- 多裝置無縫同步存取
- 不依賴單一設備或硬碟
- 按需付費，彈性擴充

📦 代表性服務

- **OneDrive** (Microsoft)：文件與照片同步
- **Google Drive** (Google)：協作文件編輯
- **Microsoft 365**：Office 軟體雲端化
- **AWS / Azure**：企業級雲端運算平台

雲端讓個人主機的概念再次轉型——運算資源重回「遠端」，但這次是服務每一個人。



第六階段

AI 時代：電腦開始理解人類

這是迄今為止最深刻的一次轉變——電腦不再只是「執行指令的工具」，而是開始「理解語言、推理思考」。

ChatGPT (OpenAI)



大型語言模型 (LLM)，能以自然語言對話、寫作、解釋程式碼，2022 年底席捲全球，僅 5 天突破百萬用戶。

Copilot (Microsoft)



整合於 Windows、Office、GitHub 的 AI 助手，讓 AI 成為日常工作流程的一部分，而非獨立工具。

AI Agent (自主代理)



下一代 AI 不只回答問題，更能自主規劃、執行多步驟任務，例如自動整理報告、操作網頁、撰寫並測試程式碼。

AI 時代最大的改變：誰學誰的語言？

過去幾十年，是人類學習電腦的語言；而現在，電腦開始學習人類的語言。這是電腦演進史上最根本的典範轉移。



以前：人學電腦語言

工程師必須用精確的程式語法與電腦溝通，任何一個錯誤的符號都會導致失敗。

```
SELECT * FROM Customer WHERE city = 'Taipei'
```

只有受過訓練的人，才能有效使用電腦。



現在：電腦學人類語言

任何人都可以用日常語言下指令，AI 自動理解意圖並執行。

「幫我分析這份銷售資料，找出業績最好的三個城市，並製作圖表。」

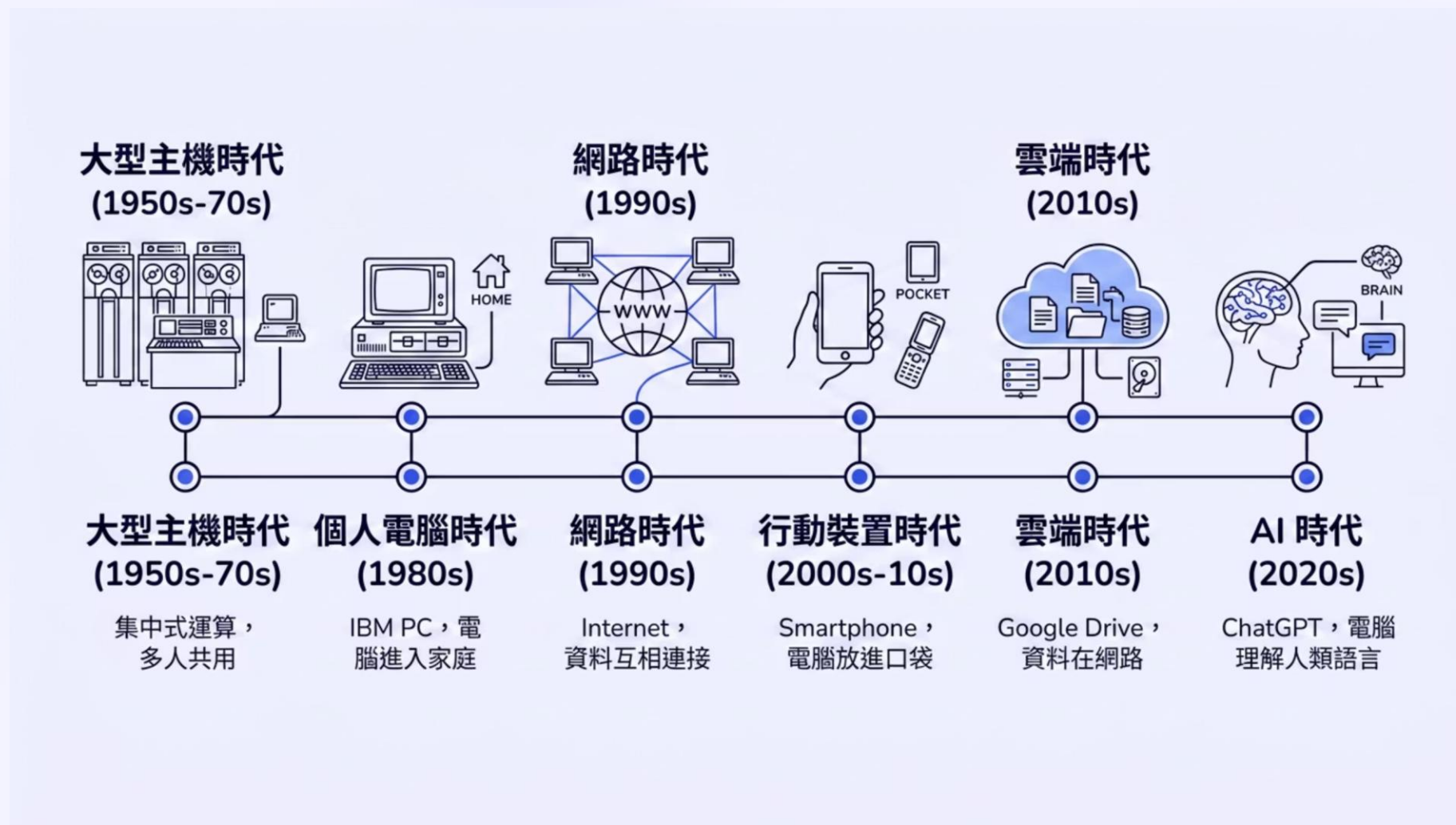
門檻大幅降低，人人都能運用 AI 的力量。



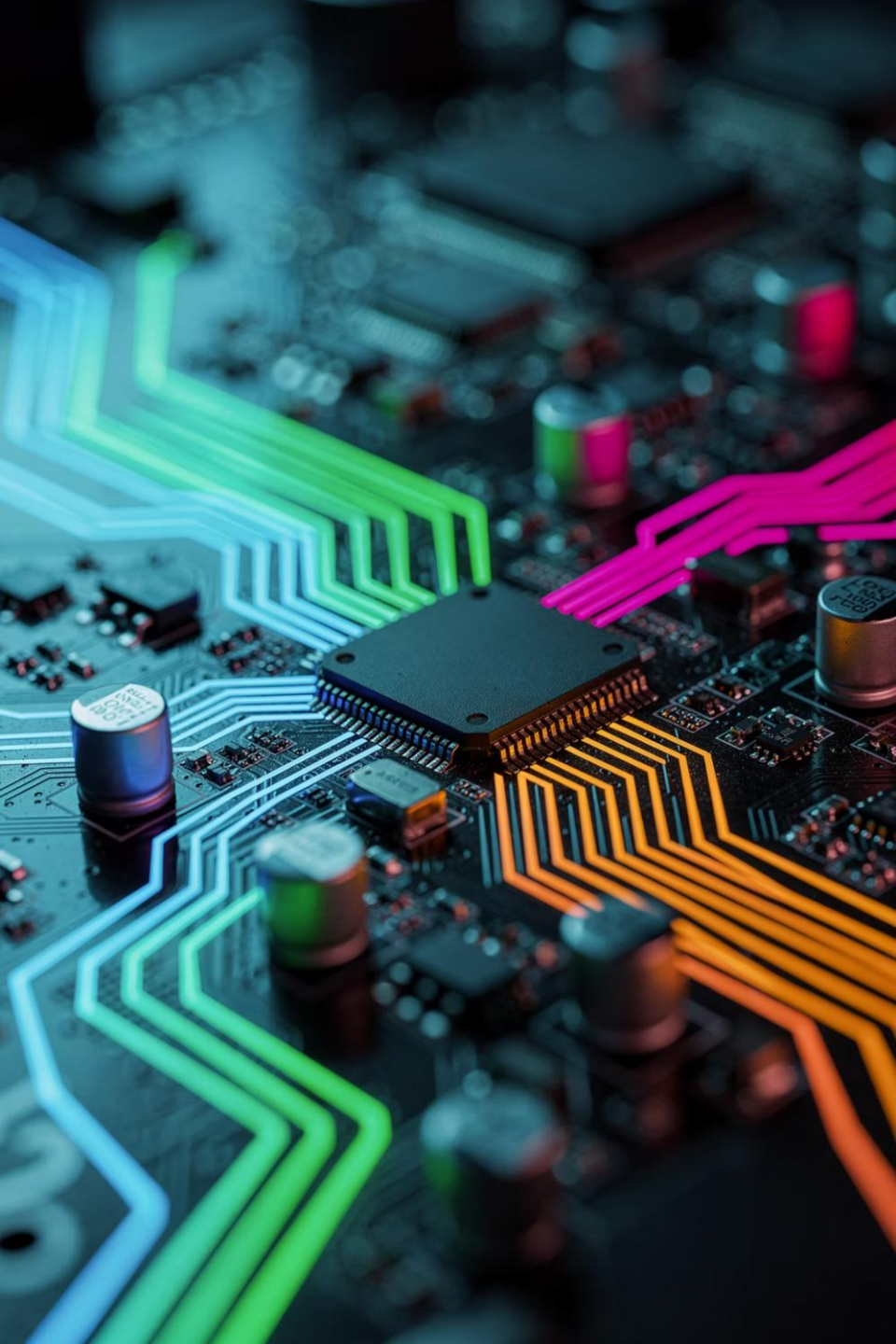
結論：人 + AI > 人 了解 AI 如何運作，讓你在 AI 時代成為更強大的使用者與開發者。

電腦演進史：完整回顧

六個階段，每一步都在改寫人與電腦的關係——從稀缺到普及，從複雜到直覺，從工具到夥伴。



下一堂課預告：我們將深入探討這些技術背後的程式設計基礎——了解電腦如何「思考」，是學習程式設計的第一步。



電腦的組成

一堂帶你看懂電腦內部世界的入門課



電腦的組成

電腦已經是我們日常生活中不可缺少的工具——從寫作業、聽音樂，到視訊通話、玩遊戲，幾乎無所不在。但你有沒有想過，這台機器到底是怎麼「思考」的？

這堂課將帶你從零開始，一步一步拆解電腦的組成、運作方式，以及它如何聽懂我們的指令。理解電腦，是學習資訊科技最重要的第一步。

入門課程

資訊科技基礎

電腦到底是什麼？

很多人對電腦有各種不同的想像，但哪一個才是正確的呢？

常見的誤解

- 電腦是超快的計算機？
- 電腦是超大的儲存空間？
- 電腦是很聰明的機器？

真正的答案

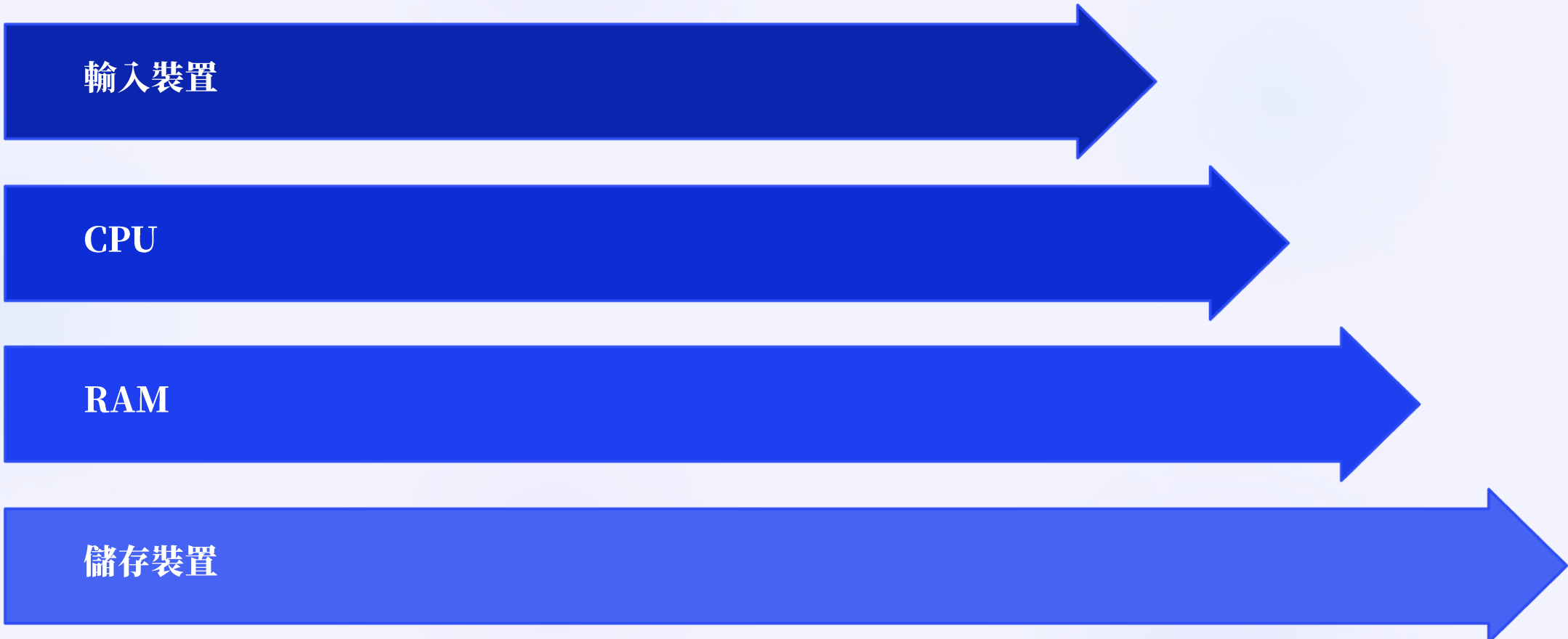
電腦其實一點都不「聰明」。它只是一台**嚴格遵守規則、依照指令執行動作**的機器。

它不會自己思考，也不會自己決定做什麼——一切都來自人類事先寫好的**程式與指令**。

 所謂的「智慧」，來自工程師精心設計的演算法，而不是電腦本身有意識。

電腦五大組成元件

一台完整的電腦，由以下五個核心部分組成，缺一不可。它們互相配合，才能讓電腦正常運作。



輸入裝置

CPU

RAM

儲存裝置

從使用者輸入資料開始，經過 CPU 處理、RAM 暫存、Storage 保存，最終透過輸出裝置呈現結果——這就是電腦工作的完整流程。

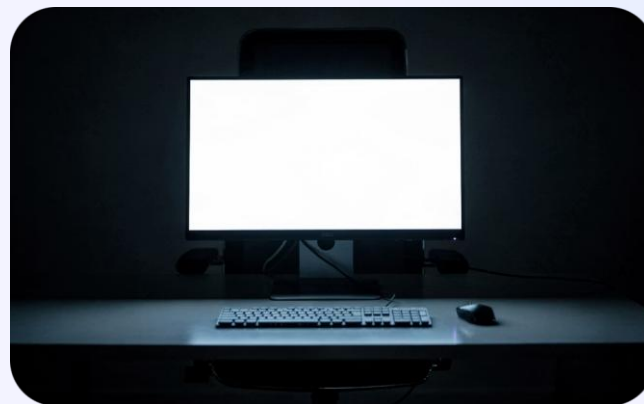
輸入與輸出裝置

輸入裝置負責把外界的資訊「送進」電腦；輸出裝置則把電腦處理後的結果「呈現」給使用者。



輸入裝置

- 鍵盤——輸入文字與指令
- 滑鼠——控制游標與點擊
- 麥克風——輸入聲音
- 攝影機——輸入影像



輸出裝置

- 螢幕——顯示文字與畫面
- 喇叭——播放聲音
- 印表機——輸出紙本資料

CPU 是什麼？

CPU (Central Processing Unit · 中央處理器) 是電腦的核心，負責執行所有的運算與邏輯判斷，並指揮其他元件如何工作。

計算

執行加減乘除等數學運算，每秒可執行數十億次。

邏輯判斷

判斷「如果...就...否則...」的條件分支，是程式流程的關鍵。

控制協調

指揮記憶體、儲存裝置與輸出裝置，讓整台電腦同步運作。

💡 比喻：CPU 就像廚師——它接收食材（資料），依照食譜（程式）烹調（處理），端出成品（結果）。



RAM 與 Storage：記憶體 vs 儲存空間

RAM（隨機存取記憶體）

- 暫時儲存正在使用的資料
- 速度極快，讓程式即時反應
- 關機後資料消失
- 容量較小（通常 8~32 GB）

 比喻：RAM = 工作桌，放你現在正在用的東西

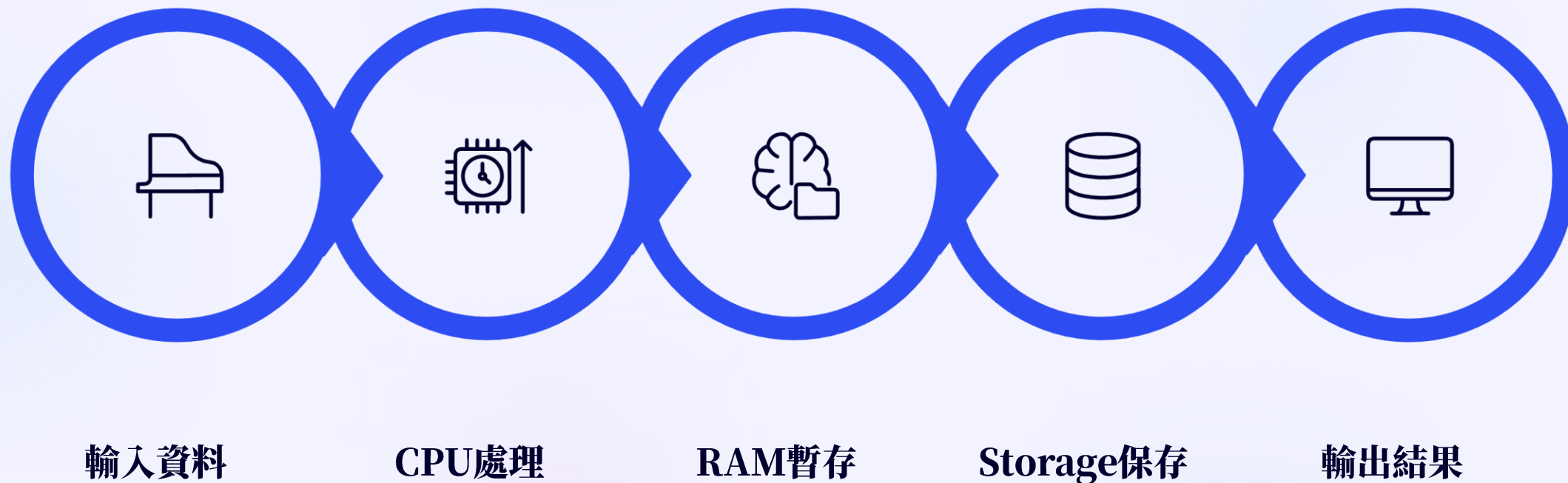
Storage（儲存裝置）

- 永久保存資料，不受電源影響
- 容量大，存放所有檔案與程式
- 速度較 RAM 慢
- 常見：SSD、HDD、USB

 比喻：Storage = 冰箱，把食材長期保存，要用再拿出來

電腦是如何工作的？

每當你對電腦下達一個指令，背後其實經歷了一連串精密的步驟。讓我們來看看完整的工作流程：



這個流程就像一條流水線：每個元件各司其職，缺少任何一個環節，電腦都無法正常運作。理解這個流程，是學習電腦科學最基礎的概念。



電腦只認識兩個數字：0 與 1

不管是文字、圖片、音樂還是影片，在電腦的世界裡，一切都被轉換成 **0** 和 **1** 的組合來儲存與處理。這種計數方式稱為 **Binary (二進位)**。

為什麼不用十進位？

電腦內部的電晶體只有兩種狀態：**通電 (1)** 或 **斷電 (0)**。這就像電燈的開與關，非常穩定、不容易出錯，因此二進位是電腦最自然的語言。

二進位的範例

十進位的 5 → 二進位為 101

十進位的 10 → 二進位為 1010

字母 A → 二進位為 01000001

電腦的最小單位：Bit 與 Byte

在電腦的世界裡，資料有其最基本的計量單位，就像長度有「公分」、重量有「公克」一樣。

1

Bit (位元)

電腦最小的資料單位，只能表示 0 或 1 其中一個值。

8

Bits = 1 Byte

8 個位元合在一起形成 1 個位元組 (Byte)，可以表示 256 種不同的值。

256

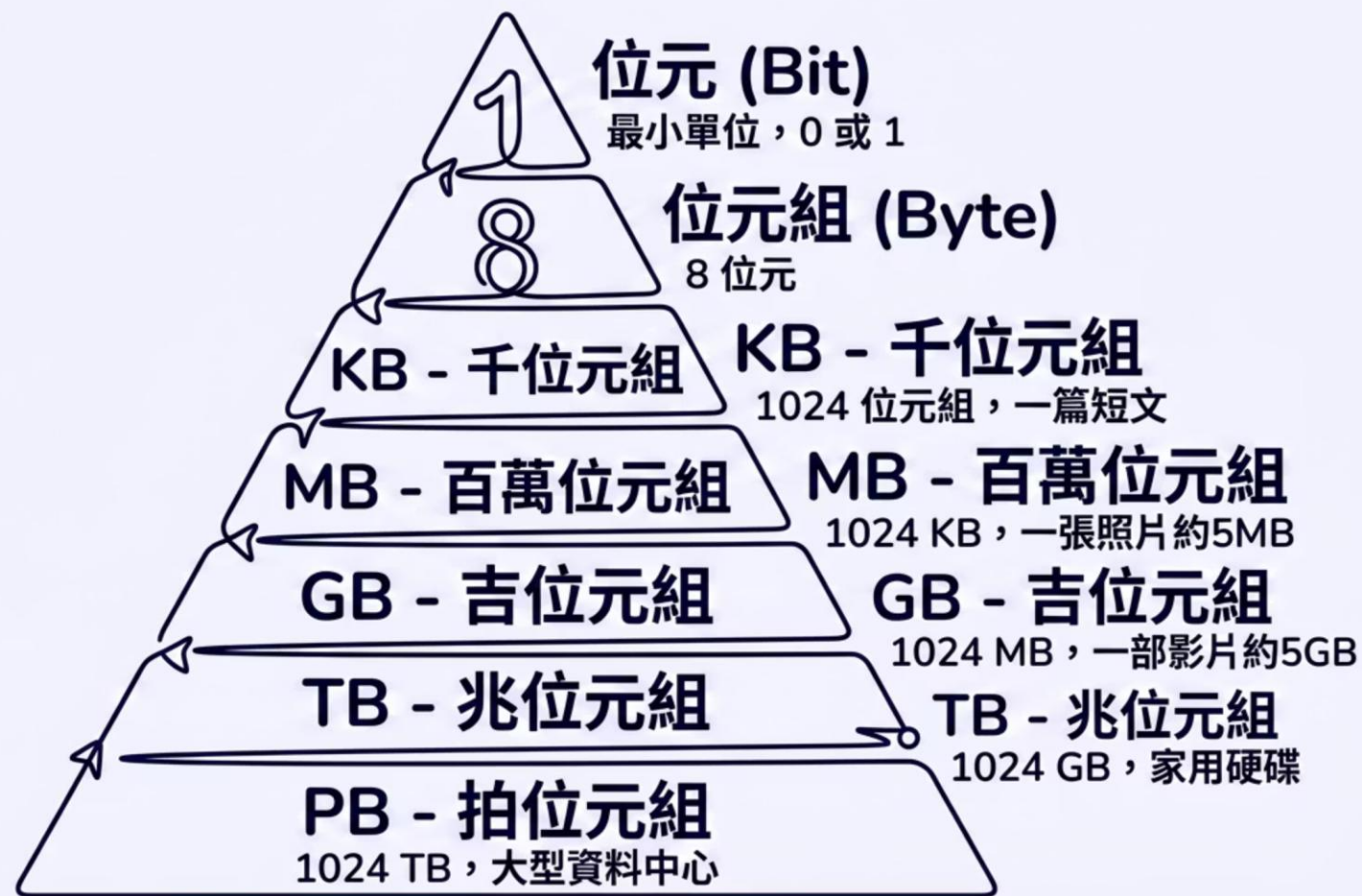
種可能組合

1 個 Byte (8 bits) 能表示 0 到 255 共 256 種不同的數字或字元。

  英文字母 A 就是用 1 個 Byte (01000001) 來表示的。

容量單位：從 KB 到 PB

隨著資料量越來越大，我們需要更大的單位來描述儲存空間。以下是常見的容量單位與換算關係：



日常生活中最常接觸的是 GB（手機儲存空間），而企業與政府的大型資料中心則可能儲存高達數百 PB 的資料。

常見的進位系統

電腦科學中除了我們熟悉的十進位，還使用兩種重要的進位系統，各有不同的用途：

1

十進位 (Decimal, DEC)

使用 **0 ~ 9** 共 10 個數字，是我們日常生活使用的計數方式。逢十進一。

範例：157、2024

2

二進位 (Binary, BIN)

只使用 **0 與 1** 兩個數字，是電腦內部唯一的計算語言。逢二進一。

範例：1010、11001

3

十六進位 (Hexadecimal, HEX)

使用 **0~9 與 A~F** 共 16 個符號，常用於表示記憶體位址與顏色代碼。逢十六進一。

範例：#FF5733 (網頁顏色)



程式語言是什麼？

這是電腦科學中最關鍵的橋梁問題——人類與電腦之間，語言完全不同，要怎麼溝通？

人類說的話

「請幫我計算這個月的薪資，並且寄出電子郵件給主管。」

清楚、自然、有語境，人類一看就懂。

電腦聽到的語言

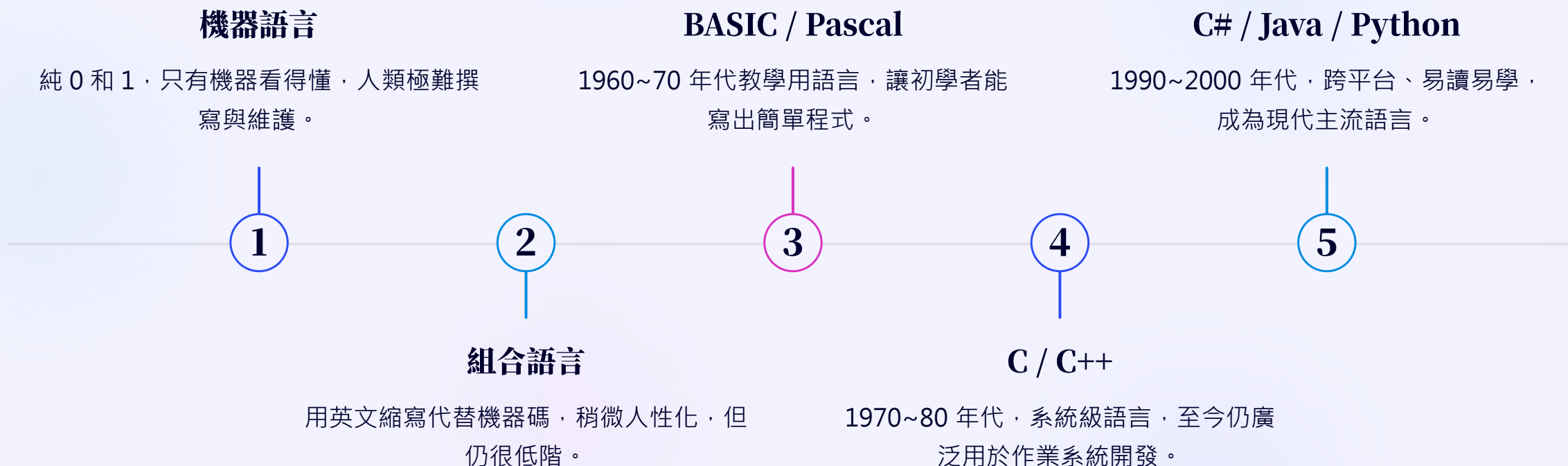
```
10100101 00110010 11001010  
01010111...
```

電腦只認識 0 和 1 組成的機器碼，完全無法理解自然語言。

📄💡 因此，人類發明了「程式語言」——一種介於人類語言與機器語言之間的橋梁，讓人可以用接近人類的方式寫出指令，再由電腦翻譯執行。

程式語言的演進歷史

從最原始的機器語言，到現代人工智慧時代的 Python，程式語言經歷了數十年的演化，每一代都讓程式設計變得更容易、更強大。



今天常見的程式語言與用途

不同的程式語言有不同的強項，工程師會根據專案需求選擇最合適的工具。以下是目前業界最常見的幾種語言：

語言	主要用途	常見應用場景
C / C++	系統開發	作業系統、遊戲引擎、嵌入式裝置
Java	企業系統	銀行後台、Android App、大型平台
C#	Windows / 企業	Windows 應用程式、Unity 遊戲開發
Python	AI / 資料分析	機器學習、資料科學、自動化腳本
JavaScript	網站開發	網頁互動效果、前端介面、Node.js
SQL	資料庫管理	查詢、新增、修改資料庫中的資料



為什麼在 AI 時代還要學程式？

這是許多人都會問的問題：既然 AI 已經可以幫我們寫程式，我們還需要學嗎？

過去：人學電腦語言

人類必須用電腦能懂的語言（程式碼）來下指令，門檻高、學習曲線陡。

現在：AI 學人類語言

AI 已能理解自然語言、自動生成程式碼，讓不懂程式的人也能創造工具。

但關鍵仍在人

人必須知道**如何拆解問題**、設定正確目標，才能引導 AI 做出有意義的事。

程式設計真正訓練的，不是打字的能力，而是**邏輯思考的能力**——這是 AI 無法替代你的核心競爭力。



程式語言與開發環境入門

從零開始，學會與電腦溝通的語言。本課程帶你認識程式語言的種類、執行原理、開發工具，以及寫下你的第一行程式碼。

初學者入門課程



程式語言與開發環境

開始與電腦溝通

電腦是強大的工具，但它只懂 0 和 1。程式語言就是我們與電腦之間的橋樑，讓人類的想法能夠被機器理解並執行。本課程將帶你一步步認識這座橋是如何搭起來的。

什麼是程式語言？

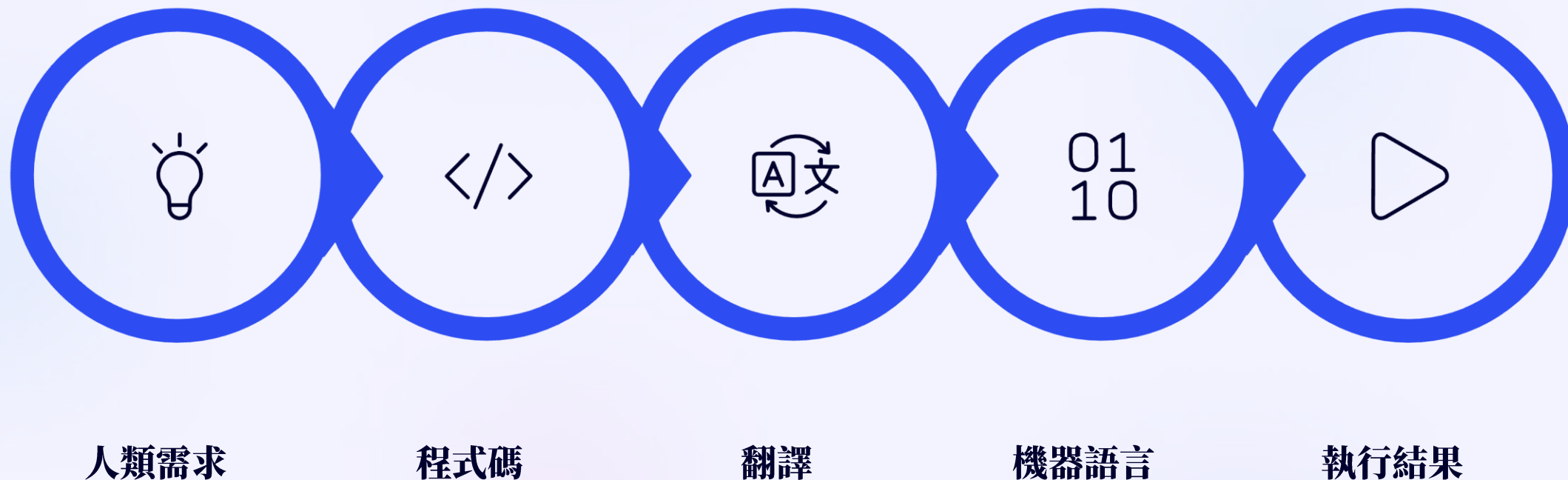
你對電腦說「幫我計算薪資」，電腦卻只懂 `101010100101`。這之間的落差怎麼解決？靠的就是**程式語言**。



程式語言就像一位**翻譯員**，讓人類能用接近自然語言的方式下達指令，再由電腦將其轉換為機器可執行的二進位碼。

程式如何執行？

從你的想法到電腦真正執行動作，中間會經歷一段清晰的流程。了解這個流程，能幫助你更直觀地理解「寫程式」這件事的本質。



每一次程式的執行，都是這條流水線在幕後默默運作。身為開發者，你的主要工作集中在「撰寫程式碼」這個環節。

程式語言分類

核心差異

兩大類程式語言的根本區別在於：翻譯的時間點不同。
是邊執行邊翻譯，還是先全部翻譯好再執行？這個問題決定了語言的特性與適用場景。

兩大主要分類

- 直譯式語言 (Interpreted)：執行時即時翻譯，靈活快速開發
- 編譯式語言 (Compiled)：先完整翻譯成可執行檔，執行效率高

理解這兩種語言的運作方式，是學習任何程式語言的重要基礎。

什麼是直譯式語言？



程式碼

即時直譯

立即執行

直譯式語言的 **Interpreter** (直譯器) 就像一位**即時口譯員**——你說一句，他翻一句，電腦馬上執行。程式碼不需要事先整體編譯，讀一行、翻一行、跑一行，開發者可以快速看到執行結果。



直譯式語言優缺點

✓ 優點

開發速度快

不需等待編譯，修改後立即可測試

容易測試與除錯

可逐行執行，快速定位問題

跨平台支援

同一份程式碼可在不同作業系統執行

X 缺點

執行速度較慢

即時翻譯需要額外的處理時間

錯誤較晚發現

部分錯誤只在執行時才會浮現

常見直譯式語言



Python

AI 開發、資料分析、自動化腳本的首選語言，
語法簡潔易讀



JavaScript

網頁前後端開發的核心語言，讓網頁擁有互動
與動態效果



PHP / Ruby

網站後端開發常用語言，廣泛應用於內容管理
與電商平台

什麼是編譯式語言？

編譯式語言採用不同策略：在執行前，先將**全部原始碼**一次翻譯完成，產生可直接執行的程式檔。



這個過程就像先把整本書從中文翻成英文，再交給讀者閱讀。翻譯需要時間，但讀起來（執行）更流暢、更快速。

編譯式語言優缺點

✓ 優點

執行速度快

已預先翻譯，執行時無需額外處理

效率高

更貼近機器語言，性能卓越

提前發現錯誤

編譯階段即可找出語法與型別錯誤

X 缺點

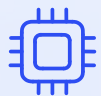
修改需重新編譯

每次改動後都需要重跑完整編譯流程

開發週期較長

等待編譯會拖慢開發與測試節奏

常見編譯式語言



C / C++

底層系統開發、嵌入式系統、高效能遊戲引擎的基石語言，執行速度極快



C#

微軟生態系的主力語言，廣泛用於 Windows 桌面程式、Unity 遊戲開發與企業系統



Go / Rust

現代系統語言的新星。Go 擅長雲端後端服務，Rust 以記憶體安全著稱

什麼是演算法？

演算法 (Algorithm) 就是「解決問題的明確步驟」。只要步驟清晰、有邏輯，任何人都能照著做出相同的結果。



核心公式：**程式 = 演算法 + 資料**

有了解題步驟 (演算法) 和處理對象 (資料)，才能構成完整的程式。

生活範例：泡麵演算法

1. 燒一鍋開水
2. 將麵條放入鍋中
3. 加入調味包
4. 等待 3 分鐘
5. 完成，可以享用

這套步驟明確、可重複、有順序——這就是演算法的精髓。

什麼是 IDE ？

IDE (整合開發環境 · Integrated Development Environment) 是將開發者所需的所有工具整合在同一個介面中的軟體，讓你不需在多個工具間切換。



程式碼編輯

語法高亮與自動補全



編譯與建置

一鍵將程式碼轉為可執行檔



除錯工具

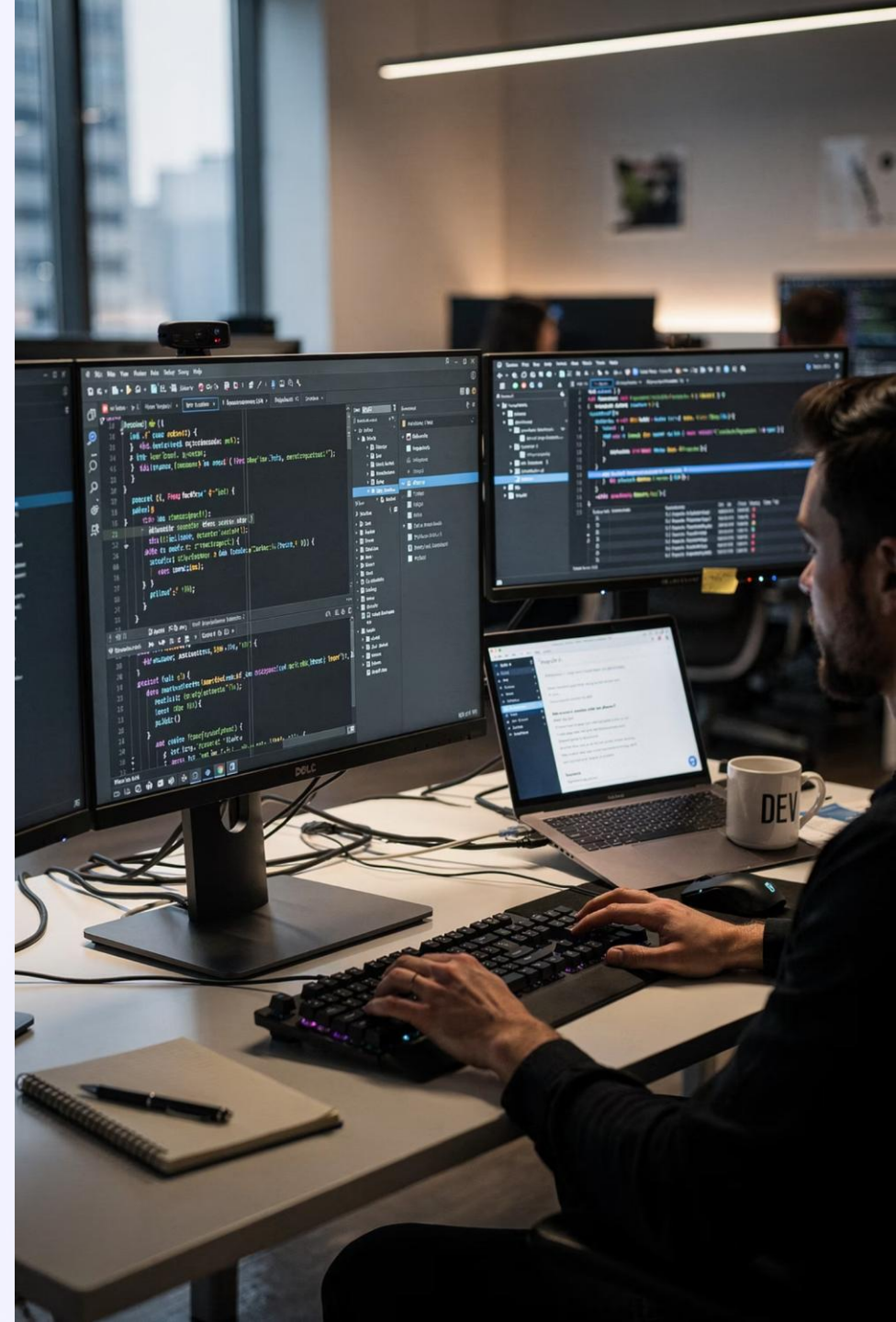
逐步追蹤程式執行狀況



直接執行

在 IDE 內即可測試程式結果

常見的 IDE 有 **Visual Studio** (C#/C++) 、 **VS Code** (多語言通用) 、 **PyCharm** (Python 專用) 等。



第一個程式：Hello World

每個程式設計師的起點，都是讓電腦說出第一句話。C# 也支援 **最上層陳述式**（**top-level statements**），所以不需要先宣告 class 或 Main 方法，直接寫程式碼就可以：

```
Console.WriteLine("Hello World");
```

執行後，畫面上會出現：`Hello World`

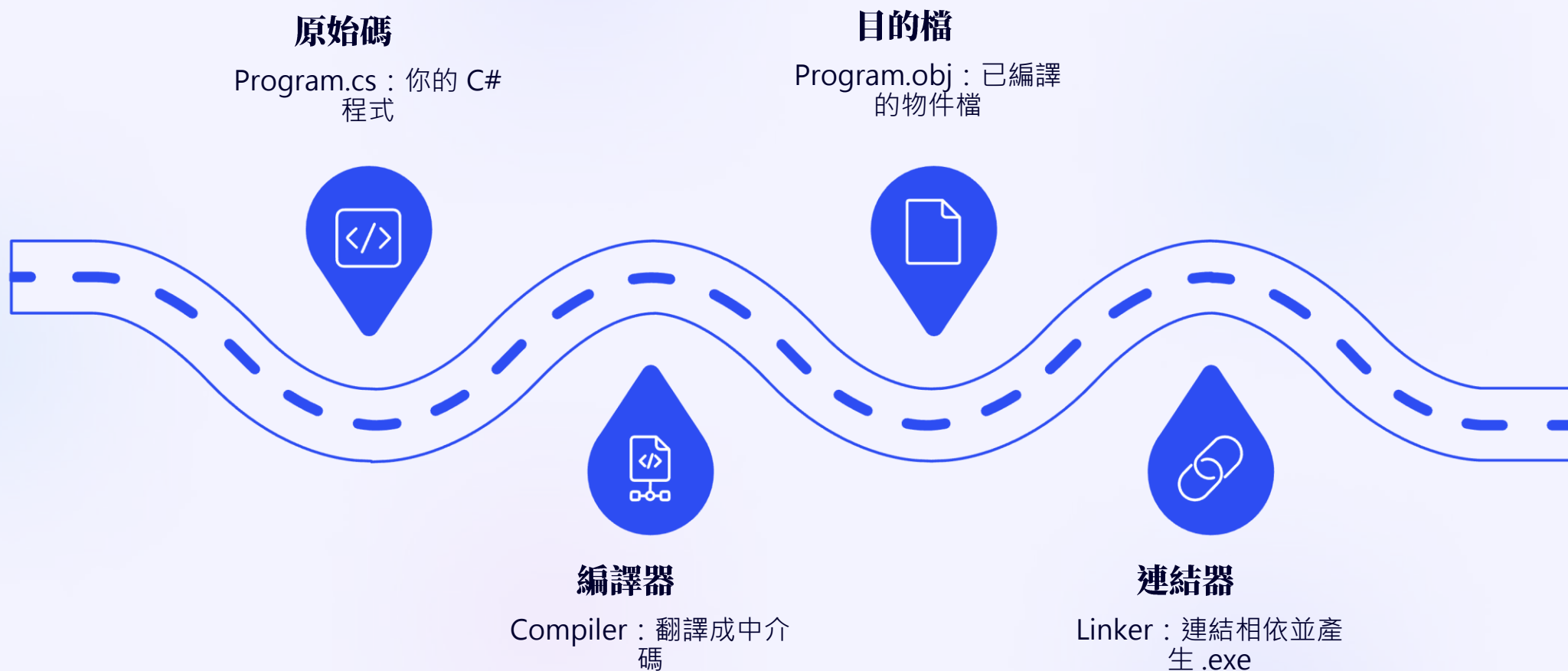
程式碼解析

`Console.WriteLine()`

輸出文字到畫面並換行

Hello World 編譯流程

寫好程式碼後，要讓電腦真正執行它，需要經過完整的編譯過程。原始碼本身無法直接被電腦執行，必須透過工具鏈轉換。



重要觀念：原始碼 ≠ 可執行檔。你在 IDE 中看到的 .cs 檔案，和最終在電腦上跑的 .exe 檔案，是完全不同的東西。

Visual Studio 專案結構

當你用 Visual Studio 建立一個 C# 專案，IDE 會自動產生一個資料夾結構。理解這個結構，能幫助你快速找到需要的檔案。

專案資料夾結構

```
CS_Solution
├─ bin
│   └─ Debug
│   └─ Release
├─ obj
│   └─ Debug
│   └─ Release
├─ Properties
├─ Ap01.csproj
├─ App.config
└─ Program.cs
```

Program.cs

你撰寫程式邏輯的主要原始碼檔案

bin 資料夾

存放最終可執行的 `.exe` 檔案
(Debug / Release 兩種版本)

obj 資料夾

編譯過程中產生的暫存目的檔，通常不需要手動修改

.csproj 檔案

專案設定檔，記錄相依套件、編譯目標等配置資訊



C# Console 基礎程式實作

從零開始學習 C# 控制台程式設計——掌握變數、輸入輸出、資料型別與演算法，打好程式設計的核心基礎。

初學者課程

C# 基礎

今天的學習目標

本次課程將帶領你一步步建立 C# 控制台程式的基礎能力，從輸入輸出到型別轉換，循序漸進地掌握每個核心觀念。

1

輸入與輸出

使用 `Console.ReadLine()` 與
`Console.WriteLine()` 與使用者互動

2

宣告變數

學習如何宣告變數並儲存程式執行中的資料

3

資料型別

認識 C# 中各種基本型別及其記憶體大小

4

型別轉換

了解型別之間的轉換規則，以及溢位 (Overflow) 問題

5

演算法實作

將問題拆解為流程圖、虛擬碼，最後寫成 C# 程式



第一個互動程式：Hello, 使用者！

讓程式學會「聆聽」使用者輸入，並給出回應。這是所有互動程式的起點。本練習將學會三個重要概念：

`Console.ReadLine()`

從鍵盤讀取一行文字輸入，並以字串型別回傳

`Console.WriteLine()`

輸出一行文字到控制台，輸出後自動換行

字串串接

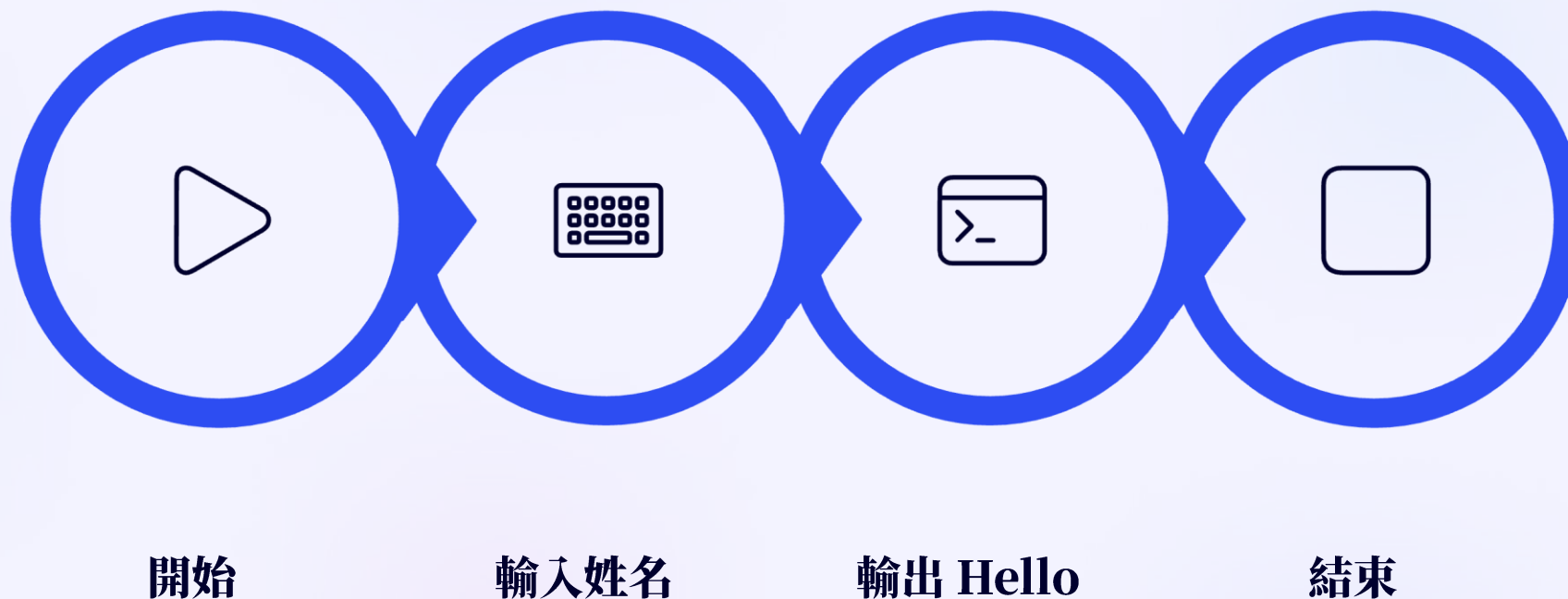
使用 `$"..."` 插值語法將變數嵌入字串，例如 `$"Hello, {name}"`



目標輸出：使用者輸入 **Jerry**，程式顯示 **Hello, Jerry**

流程圖：Hello 程式執行步驟

在撰寫程式碼之前，先用流程圖釐清程式的執行邏輯，是良好的程式設計習慣。每一個步驟都代表程式中的一個動作。



流程圖幫助我們在寫程式前先思考「程式要做什麼」，讓邏輯更清晰、出錯機率更低。

Hello World 程式碼解析

程式碼

```
Console.Write("請輸入姓名 : ");  
string name = Console.ReadLine();  
Console.WriteLine($"Hello, {name}");
```

第 1 行：提示使用者輸入，不換行

第 2 行：讀取輸入並存入變數 `name`

第 3 行：使用字串插值輸出結果

執行結果



輸入：Jerry

輸出：Hello, Jerry

重點筆記

- `Console.Write()` 不會換行，適合放在同行提示文字
- `string` 是儲存文字的資料型別
- `$"..."` 字串插值讓串接更簡潔易讀
- 大小寫要完全正確，C# 區分大小寫

資料型別與記憶體空間

C# 是強型別語言，每個變數都有明確的型別。不同型別在記憶體中佔用的空間不同，能儲存的數值範圍也不同。選對型別，程式才能正確且有效率地運作。



整數型別

- `byte` : 1 byte (0 ~ 255)
- `short` : 2 bytes
- `int` : 4 bytes (最常用)
- `long` : 8 bytes (超大整數)



浮點與小數型別

- `float` : 4 bytes (單精度)
- `double` : 8 bytes (雙精度)
- `decimal` : 16 bytes (高精度金融計算)



💡 思考問題：為什麼不同型別佔用的記憶體空間不同？這會影響程式效能嗎？

使用 `sizeof()` 查詢型別大小

`sizeof()` 是 C# 提供的內建運算子，可以在執行時期或編譯時期查詢指定資料型別所佔用的 **Bytes** 數量，幫助我們理解記憶體配置。

程式碼範例

```
Console.WriteLine(sizeof(byte));  
Console.WriteLine(sizeof(short));  
Console.WriteLine(sizeof(int));  
Console.WriteLine(sizeof(long));  
Console.WriteLine(sizeof(float));  
Console.WriteLine(sizeof(double));  
Console.WriteLine(sizeof(decimal));
```

輸出結果

4

`sizeof(int)`

int 型別佔用 4 個 Bytes，可儲存約 ±21 億的整數

8

`sizeof(double)`

double 型別佔用 8 個 Bytes，提供高精度浮點數

使用 MinValue 與 MaxValue

每種整數型別都有其能儲存的最小值與最大值。C# 提供了 `.MinValue` 與 `.MaxValue` 屬性，讓我們不需要背誦這些數字，直接讓程式告訴我們答案。

程式碼

```
Console.WriteLine(int.MinValue);  
Console.WriteLine(int.MaxValue);  
Console.WriteLine(long.MinValue);  
Console.WriteLine(long.MaxValue);
```

執行結果 (int)

✓ `int.MinValue = -2,147,483,648`
`int.MaxValue = 2,147,483,647`

記憶技巧：`int` 使用 32 個 bits (4 bytes)，其中 1 個 bit 用來表示正負號，因此可表示 $2^{32} =$ 約 42 億個數字，分布在正負兩側。

各型別記憶體空間比較

下表整理了 C# 常見基本型別的記憶體佔用大小。型別越大，可儲存的數值範圍越廣，但也消耗更多記憶體資源。請根據實際需求選擇合適的型別。

型別	大小 (Bytes)	說明
byte	1	無符號整數，範圍 0 ~ 255
short	2	有符號短整數，約 ±3.2 萬
int	4	最常用整數型別，約 ±21 億
long	8	大整數，適合天文數字
float	4	單精度浮點數，約 7 位有效數字
double	8	雙精度浮點數，約 15 位有效數字
decimal	16	最高精度，專為金融財務計算設計

💡 為什麼 decimal 需要 16 Bytes ? 因為它採用十進位精確表示法，避免二進位浮點數的精度誤差，是處理金錢計算的最佳選擇。

十進位與十六進位

電腦內部以二進位儲存資料，但開發者常用十六進位（Hexadecimal）來更緊湊地表示記憶體內容。每個十六進位數字代表 4 個 bits，讓記憶體內容更易閱讀。

數字對照範例

十進位

245 (一般我們日常使用的數字)

十六進位

5FBE (以 0-9 與 A-F 共 16 個符號表示)

C# 轉換語法

```
int value = 245;  
// 輸出十六進位  
Console.WriteLine(  
    value.ToString("X"));
```

格式碼 "X" 代表大寫十六進位

格式碼 "x" 代表小寫十六進位



輸出結果：F5

練習：正負數的十六進位表示

同一個數字，正數與負數在記憶體中的十六進位表示有什麼差別？這個練習讓你觀察補數 (Two's Complement) 的實際效果，這是電腦儲存負數的核心原理。

程式碼

```
int pos = 24510;
int neg = -24510;
Console.WriteLine(pos);
Console.WriteLine(neg);
Console.WriteLine(
    pos.ToString("X"));
Console.WriteLine(
    neg.ToString("X"));
```

觀察輸出

表示方式	正數	負數
十進位	24510	-24510
十六進位	5FBE	FFFFA042

- ❏ 負數的十六進位看起來很不同！這是因為電腦用「二補數」表示法儲存負數，高位元的 F 表示最高位為 1，代表負數符號。

型別轉換問題：從 int 轉 short

當我們把一個大型別的值強制轉換為小型別時，如果數值超出目標型別的範圍，就會發生**型別溢位 (Overflow)**。讓我們來看看會發生什麼事：

程式碼

```
int b = 100000;  
short a = (short)b;  
  
Console.WriteLine($"b = {b}");  
Console.WriteLine($"a = {a}");
```

`(short)` 是強制型別轉換語法，告訴編譯器「我知道可能會有問題，仍強制轉換」

思考問題

⚠️ 問題：b = 100000，但 short 的最大值只有 32767，超過了！
請問 a 的結果會是多少？

- 100000 超過 short 的最大值 32767
- 強制轉換只保留低 16 bits
- 多餘的 bits 會被截斷丟棄
- 結果將是一個意外的數值

型別溢位 (Overflow) 解析

強制型別轉換超出範圍時，C# 不會報錯，而是靜默地截斷資料，這是程式設計中常見的隱性 Bug 來源，必須特別注意！

執行結果

⊗ `b = 100,000`
`a = -31072` (非預期結果！)

原因分析

- `short` 範圍：-32768 ~ 32767
- 100000 超出上限 32767
- 高位元 bits 被截斷丟棄
- 剩餘 bits 被解讀為負數

記憶體中的十六進位

```
b = 100000
b (hex) = 186A0

a (hex) = 86A0
a (decimal) = -31072
```

可以看到 `int` 的 `1` (最高位) 被截掉了，只保留了後 4 個十六進位數字 `86A0`，而 `8...` 開頭表示最高位為 1，代表負數。

查看十六進位了解記憶體變化

透過 `ToString("X")` 觀察轉換前後的十六進位值，可以清楚看到記憶體中資料的截斷過程，這是理解型別溢位最直觀的方式。

程式碼

```
int b = 100000;
short a = (short)b;

Console.WriteLine(
    $"b (十進位) = {b}");
Console.WriteLine(
    $"b (十六進位) = {b.ToString("X")}");
Console.WriteLine(
    $"a (十進位) = {a}");
Console.WriteLine(
    $"a (十六進位) = {a.ToString("X")}");
```

輸出結果對照

	十進位	十六進位
b (int)	100000	186A0
a (short)	-31072	86A0

📌 觀察重點：`int` 的 `186A0` 截掉最高位的 `1`，只剩下 `86A0`，這就是 `short` 實際儲存的值。

演算法實作：兩數相加

學會輸入輸出後，讓我們實作第一個真正的計算演算法：讓使用者輸入兩個整數，程式計算並輸出總和。這個看似簡單的任務，包含了程式設計的完整思路。

1

步驟 1：輸入 a

提示使用者輸入第一個數字 `a`

2

步驟 2：輸入 b

提示使用者輸入第二個數字 `b`

3

步驟 3：計算總和

執行 `sum = a + b` 計算結果

4

步驟 4：顯示結果

以格式化字串輸出 `a + b = sum`

⚠️ 注意：`Console.ReadLine()` 回傳的是字串，不能直接做數學運算！必須先用 `int.Parse()` 轉換為整數型別。

流程圖、虛擬碼與 C# 完整實作

流程圖



虛擬碼 (Pseudocode)

```
INPUT a
INPUT b
sum = a + b
OUTPUT sum
```

C# 完整程式碼

```
string s;
int a, b;

Console.Write("a = ");
s = Console.ReadLine();
a = int.Parse(s);

Console.Write("b = ");
s = Console.ReadLine();
b = int.Parse(s);

Console.WriteLine(
    $"{a} + {b} = {a + b}");
```

✓ 輸入 a=3 · b=5
輸出：3 + 5 = 8

關鍵：int.Parse(s) 將字串轉換為整數，才能進行數學運算。